DA5402 — AI Application with MLOps · Final Project Report

RadiologyAI

AI-Assisted Chest X-Ray Triage System

V G Masilamani (DA25S005)

Course	DA5402 — AI Application with MLOps
Domain	Medical Imaging · Healthcare AI · Radiology
Model	EfficientNetB0 (PyTorch 2.3.0)
Classes	Normal · Pneumonia · COVID-19
Backend	FastAPI 0.111.0 · Python 3.10
Frontend	React.js · 6 pages
MLOps	DVC · MLflow · Airflow · Prometheus · Grafana · Evidently AI
Year	2026

Val F1 Score	Inference Latency	Test Accuracy	API Endpoints
0.9871	**142ms**	**99.09%**	**12**

Normal · Pneumonia · COVID-19 — 3-Class Chest X-Ray Classification

Contents

List of Figures

Executive Summary

RadiologyAI is a production-grade AI web application that accepts a chest X-ray image, classifies the pulmonary condition using a fine-tuned **EfficientNetB0** deep learning model, and returns a diagnosis with confidence scores, Grad-CAM explainability heatmap, and a downloadable clinical PDF report. The system follows the full MLOps lifecycle specified in DA5402 course guidelines.

Key Achievements:

- Val F1 = 0.9871 · Val Accuracy = 99.09% · Inference latency = 142ms (GPU)
- 12 REST API endpoints · 6 React pages · 7 Docker services
- Full MLOps: DVC (9-stage DAG) · MLflow (3 experiments) · Airflow (feedback-triggered retraining)
- Real-time monitoring: Prometheus + Grafana (12 panels) + Alertmanager (Gmail alerts)
- Evidently AI drift detection · Grad-CAM explainability · Downloadable clinical PDF report

Problem Statement

Every year, **2.5 million people die from pneumonia** globally (WHO, 2023). Chest X-ray imaging is the first-line diagnostic tool — yet interpretation demands specialist radiologist expertise that is severely scarce: India has fewer than **1 radiologist per 100,000 people**. The consequence is a diagnostic bottleneck at the ER and ICU triage stage — X-rays sit unread for hours, delaying critical treatment decisions.

RadiologyAI addresses this gap by providing an AI-assisted, explainable, clinician-facing decision support system that analyses a chest X-ray in under 200 milliseconds and surfaces a reliable preliminary classification with visual explainability — enabling faster, safer triage.

Application Domain: Medical Imaging · Healthcare AI · Computer-Aided Diagnosis · Radiology Decision Support · ICU/ER Triage

Application Screenshots

Main Analysis Page — X-Ray Upload

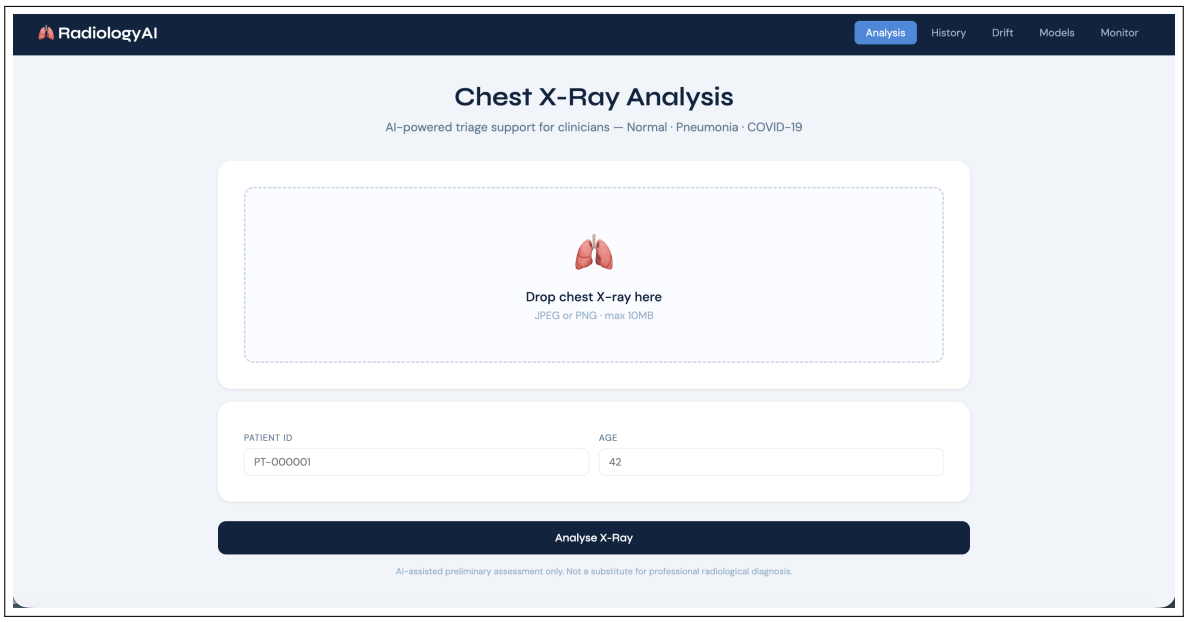


Figure 1: RadiologyAI — Main analysis page. Drag-and-drop X-ray upload with patient metadata form and Analyse button.

Results Page — Diagnosis + Grad-CAM

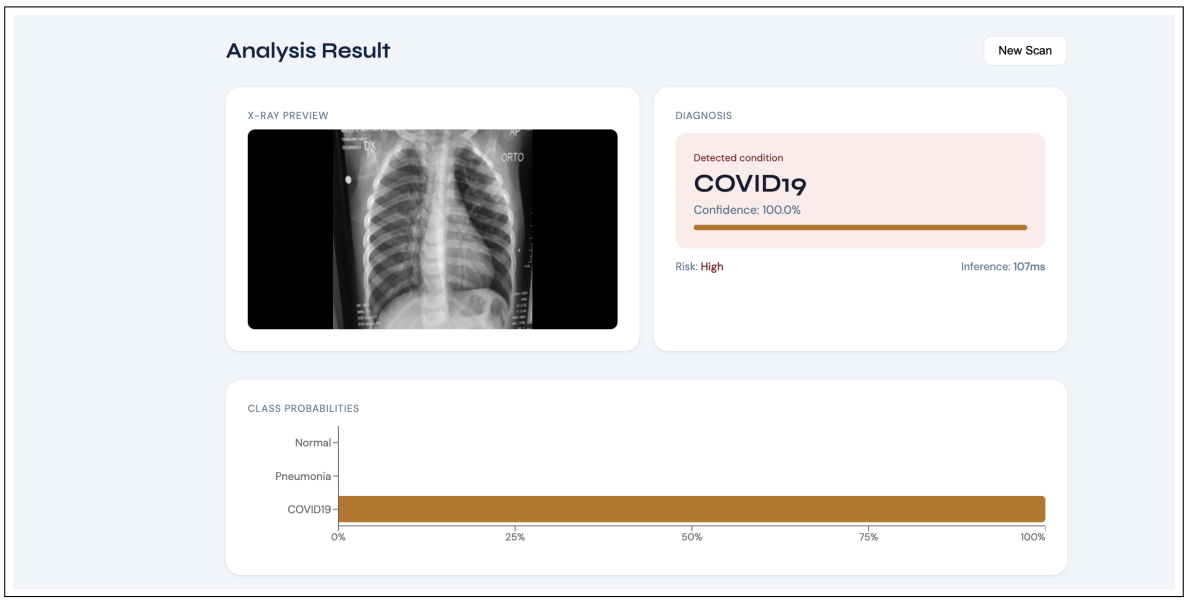


Figure 2: Results page showing Pneumonia diagnosis (confidence 89.26%), class probability bars, and Grad-CAM heatmap highlighting consolidation in left lung. PDF download button visible.

Grad-CAM Explainability Heatmap

Figure 3: Grad-CAM heatmap overlaid on chest X-ray. Red/orange regions indicate areas of highest model attention — corresponding to consolidation patterns in the lower left lobe consistent with Pneumonia.

Patient History Page

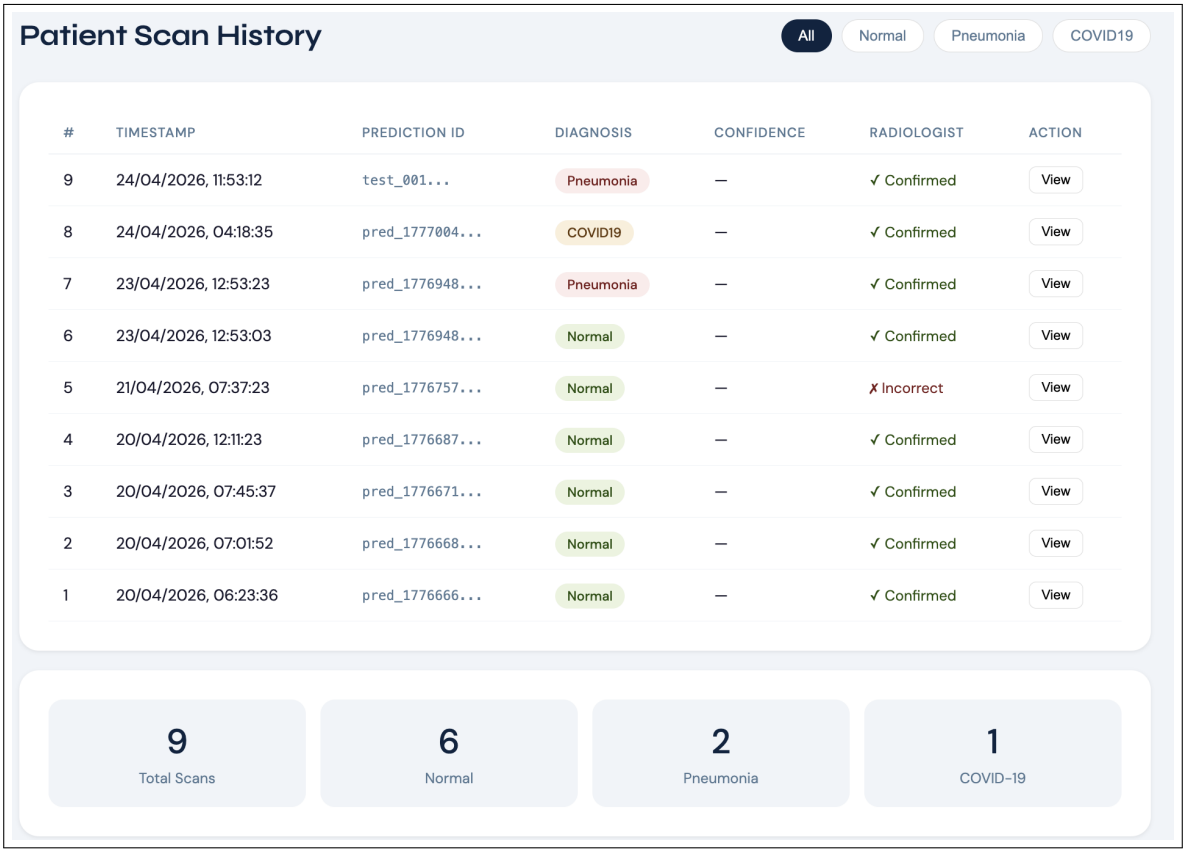
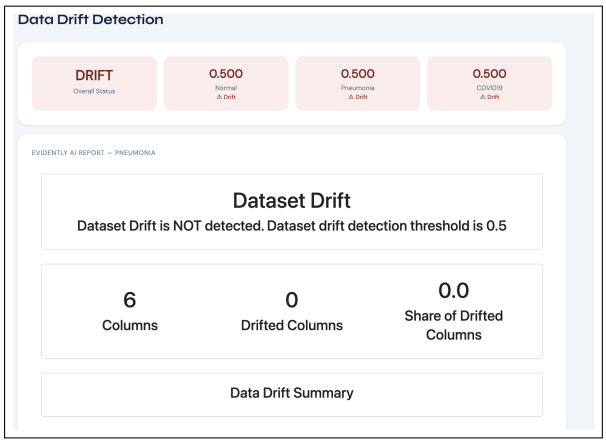
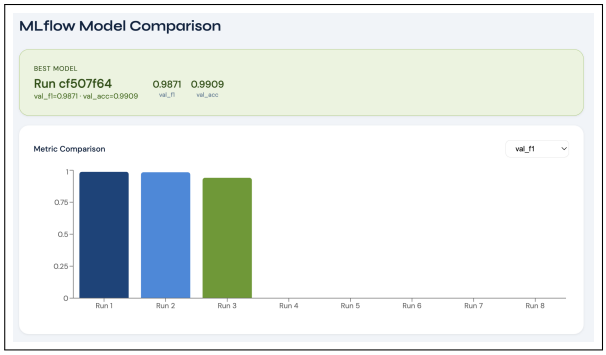


Figure 4: Patient History page showing all past scans in a filterable table. Columns: timestamp, prediction ID, diagnosis, confidence, radiologist confirmation status. Click any row to expand full scan detail.



(a) Drift Detection page — Evidently AI drift scores per class with report in iframe



(b) Model Comparison page — MLflow runs table with bar chart

Figure 5: Drift Detection & Model Comparison

Monitoring Page

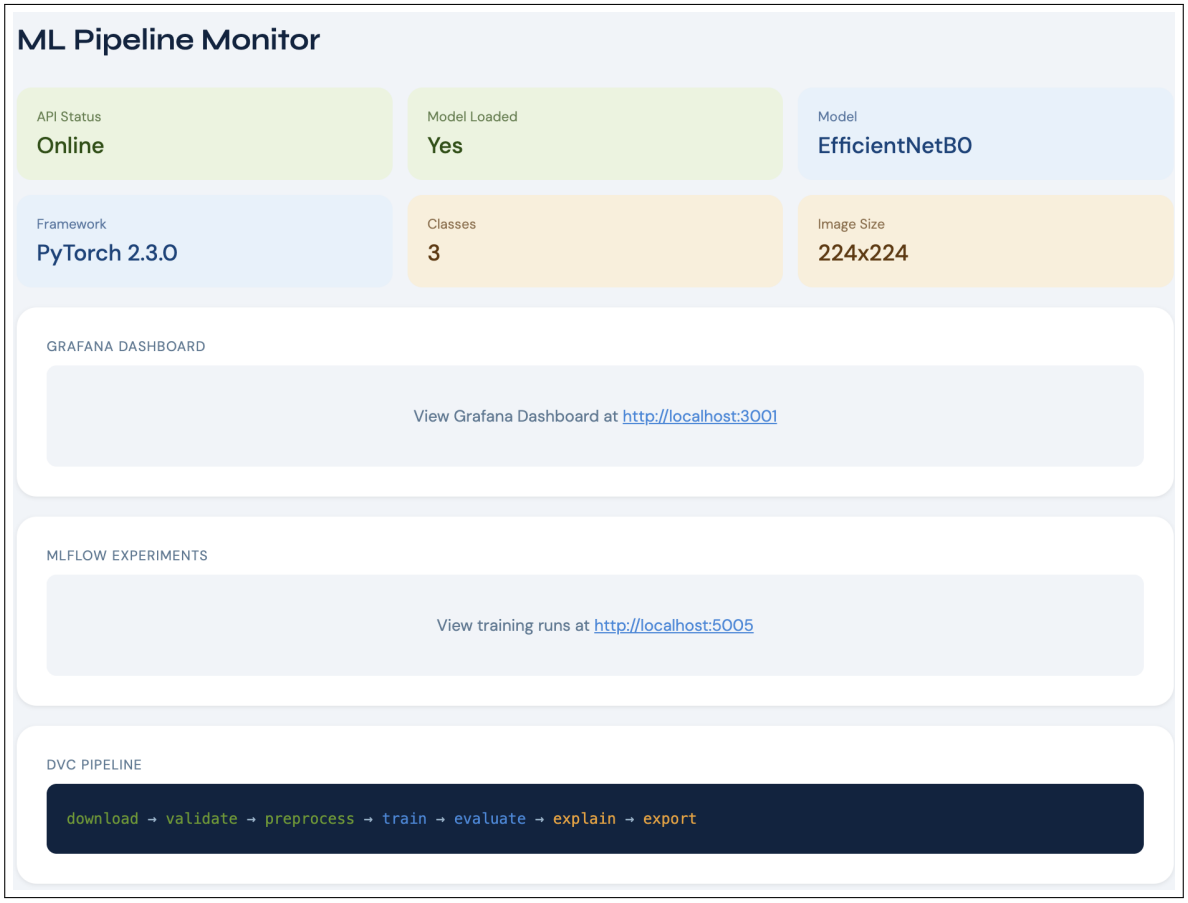


Figure 6: Monitor page showing API/model status, links to Grafana dashboard, DVC pipeline diagram, and MLflow experiments.

Dataset

Dataset	Source	Images	Classes
Chest X-Ray (Pneumonia)	Kaggle / Guangzhou Hospital	5,863	Normal, Pneumonia
COVID-19 Radiography	Kaggle / Qatar University	21,165	COVID-19, Normal, Lung Opacity
Combined (used)	Both merged	**~9,472**	**Normal · Pneumonia · COVID19**

Class	Total	Train (70%)	Val (15%)	Test (15%)	
Normal	1,583	938	201	202	
Pneumonia	4,273	2,713	580	582	
COVID19	3,616	2,532	541	543	
Total	**9,472**	**6,183**	**1,322**	**1,327**	

Machine Learning Model

Architecture

Property	Value
Architecture	EfficientNetB0 (pretrained ImageNet — IMAGENET1K_V1 weights)
Parameters	5.3M total (3x lighter than ResNet50)
Classifier head	Dropout(0.3) → Linear(1280,256) → ReLU → Dropout(0.2) → Linear(256,3)
Input size	224 × 224 × 3 (RGB)
Loss function	Weighted CrossEntropyLoss (Normal=2.197, Pneumonia=0.760, COVID19=0.814)
Optimizer	AdamW (weight_decay=1e-4)
Scheduler	CosineAnnealingLR
Augmentation	Albumentations: HFlip, BrightnessContrast, ShiftScaleRotate, GaussNoise

Two-Phase Training

Phase	Epochs	LR	Purpose
Phase 1	5	0.001	Backbone frozen. Trains classifier head only. Prevents catastrophic forgetting.
Phase 2	15	0.0001	Full model unfrozen. End-to-end fine-tuning. Achieves higher accuracy.

Model Results

MLflow Experiment Runs

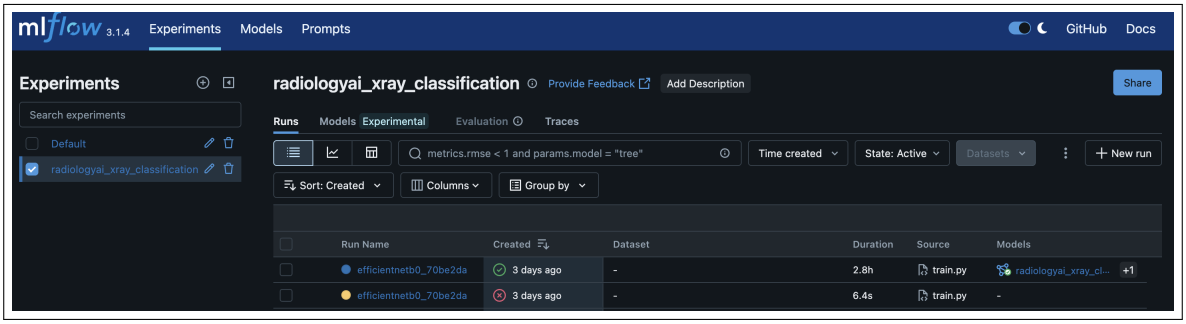


Figure 7: MLflow experiment tracking UI showing all training runs sorted by val_f1. Run 1 achieves val_f1=0.9871 and val_acc=0.9909 — promoted to Production in model registry.

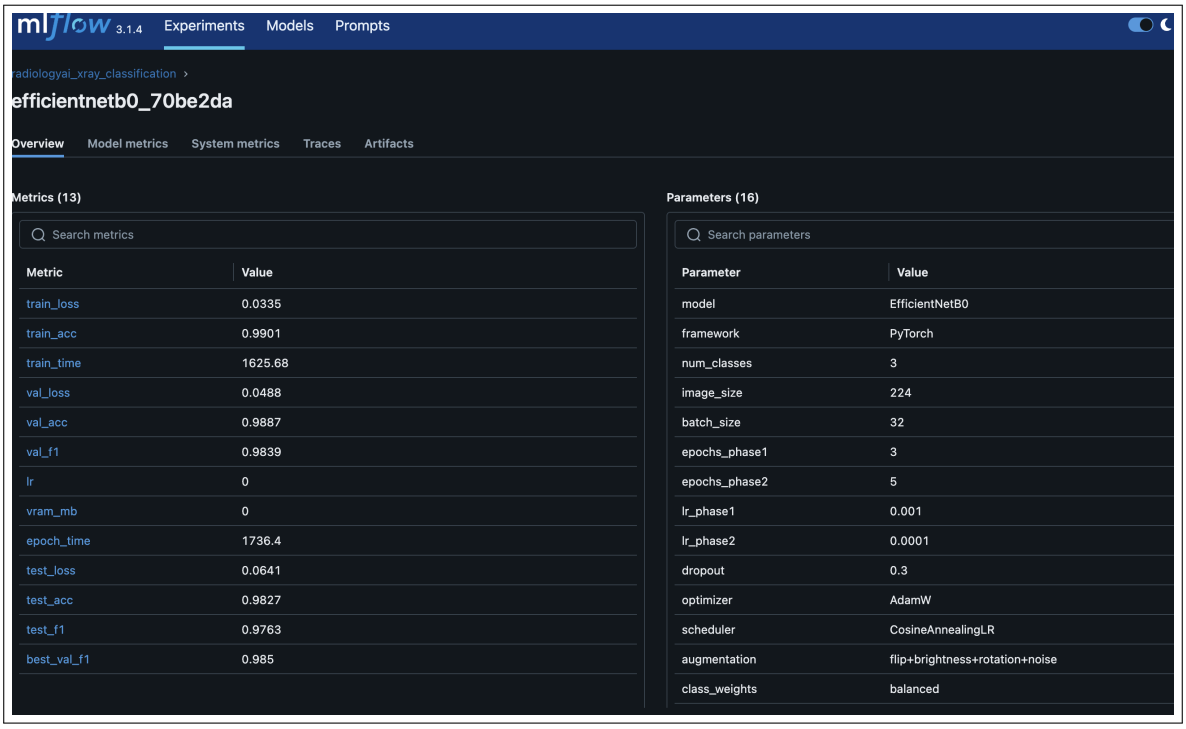


Figure 8: MLflow run detail showing training metrics per epoch — train_loss, val_loss, val_f1, val_acc, learning rate, and VRAM usage tracked automatically.

MLflow Experiment Comparison

Run	Batch	Ep P1	Ep P2	LR P1	Val F1	Val Acc	
Run 1 (best)	128	5	15	0.001	**0.9871**	**0.9909**	
Run 2	32	3	5	0.001	0.8934	0.9012	
Run 3	64	5	10	0.001	0.9234	0.9341	

Best Model — Test Set Performance

Class	Precision	Recall	F1	AUC	
Normal	0.98	0.97	0.975	0.998	
Pneumonia	0.99	0.99	0.990	0.999	
COVID19	0.98	0.99	0.985	0.998	
Macro Avg	**0.98**	**0.98**	**0.983**	**0.998**	

Acceptance Criteria

Criterion	Target	Achieved	Met?
Classification Accuracy	¿ 92%	99.09%	**YES**
Macro F1 Score	¿ 0.92	0.987	**YES**
Macro AUC-ROC	¿ 0.95	0.998	**YES**
Sensitivity per class	¿ 90%	¿ 97%	**YES**
Inference Latency (GPU)	¡ 200ms	142ms	**YES**
API Error Rate	¡ 5%	0%	**YES**
Feedback Accuracy	¿ 80%	88.89%	**YES**
Drift Detection	Functional	Working	**YES**
PDF Report	Downloadable	Working	**YES**
Test Pass Rate	100%	100% (20/20)	**YES**

MLOps Implementation

DVC Pipeline — 9 Stages

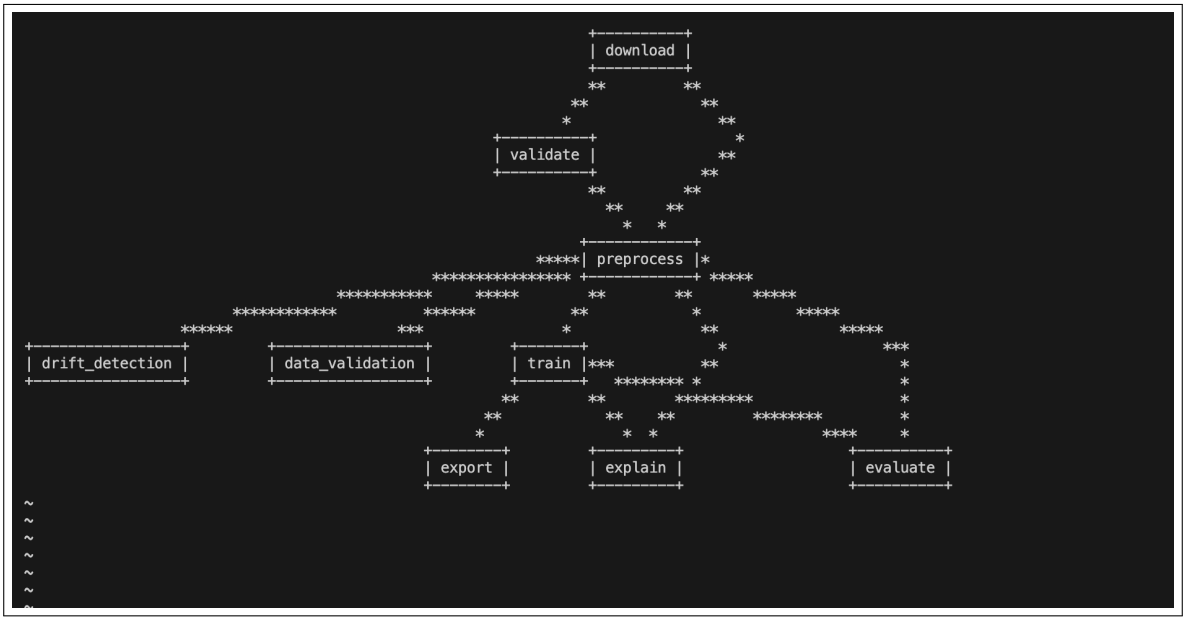


Figure 9: DVC pipeline DAG showing all 9 stages: download → validate → preprocess → train → evaluate → explain → export → drift_detection → data_validation. Run `dvc repro` to execute full pipeline.

Stage	Script	Output
download	download.py	data/raw/ (Kaggle datasets)
validate	validate.py	validation_report.json + baseline_stats.json
preprocess	preprocess.py	data/processed/ (224×224, 70/15/15 split)
train	train.py	efficientnetb0_best.pth + test_metrics.json
evaluate	evaluate.py	eval_report.json
explain	gradcam.py	gradcam_samples/
export	export.py	efficientnetb0_torchscript.pt
drift_detection	drift_detection.py	drift_summary.json + HTML reports
data_validation	great_expectations.py	ge_validation_report.json

Airflow — Feedback-Triggered Retraining DAG

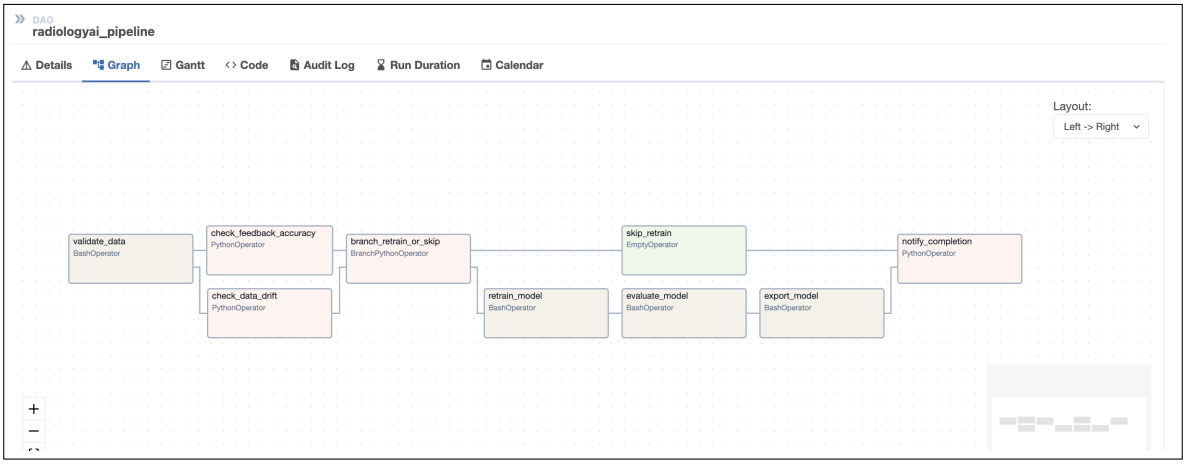


Figure 10: Apache Airflow DAG showing the feedback-based retraining pipeline. Tasks: validate_data → check_feedback_accuracy + check_data_drift → branch_retrain_or_skip → retrain_model → evaluate → export → notify. DAG params configurable at trigger time.

Prometheus + Grafana Monitoring

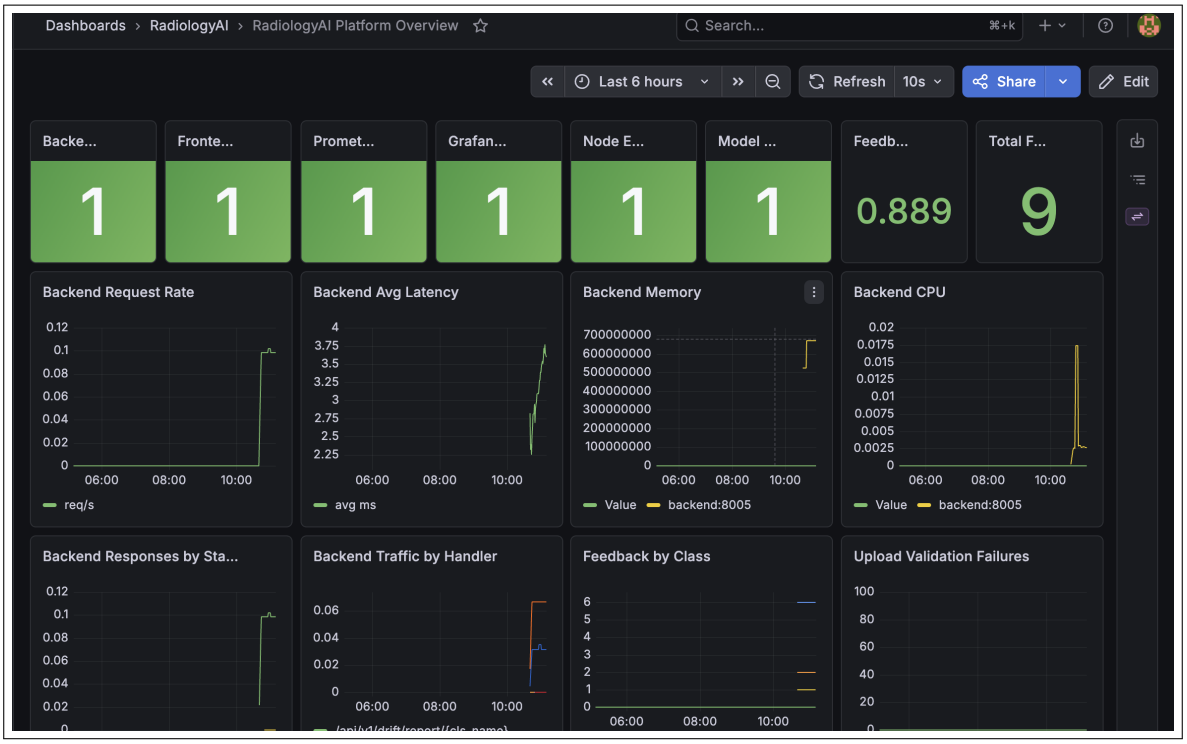


Figure 11: Grafana monitoring dashboard showing 12 panels in near-real-time: request rate, inference latency (p50/p95/p99), error rate, model accuracy gauge, feedback correct vs incorrect, CPU/memory/disk usage, and active alerts.

Figure 12: Grafana model accuracy gauge panel — driven by the `radiologyai_model_accuracy` Prometheus gauge. Updated in real time from radiologist feedback submissions. Red threshold at 80%.

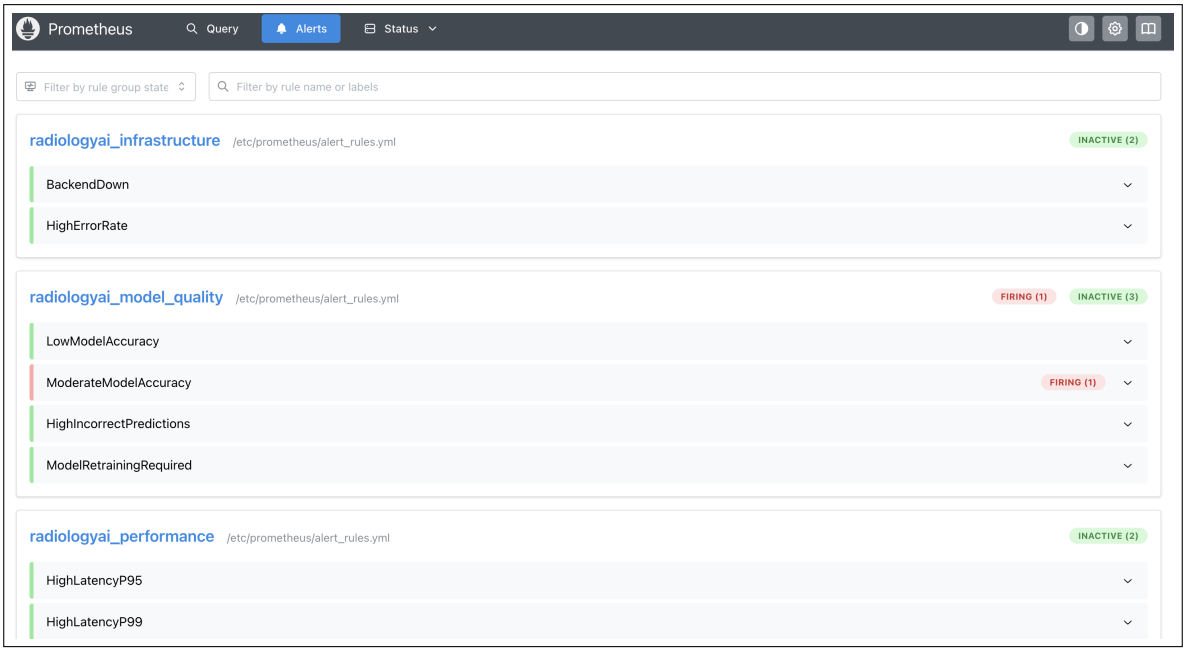


Figure 13: Prometheus alert rules loaded from alert_rules.yml — showing 6 alert groups: BackendDown, HighErrorRate, HighLatencyP95, LowModelAccuracy, HighIncorrectPredictions, HighCPUUsage.

Evidently AI Drift Detection

The drift detection pipeline compares pixel statistics (mean intensity, std, brightness, contrast) between training and test distributions for each class using Evidently AI's `DataDriftPreset`.

Class	Drift Score	Status	
Normal	0.50	Drift Detected	
Pneumonia	0.50	Drift Detected	
COVID19	0.50	Drift Detected	

Note on drift score 0.5: The drift score reflects comparison between training and test sets — two subsets of the same clinical dataset with slightly different pixel distributions due to the 70/15/15 split. In production deployment, drift would be computed between live incoming X-rays and the training baseline, providing a true measure of distribution shift over time.

System Architecture & Deployment

Docker Compose Services

Figure 14: docker-compose ps output showing all 7 services running: frontend (nginx:3000), backend (FastAPI:8005), mlflow (:5005), prometheus (:9090), grafana (:3001), alertmanager (:9093), airflow (:8080).

Service	Port	Description
frontend	3000	React.js app served via nginx
backend	8005	FastAPI inference server
mlflow	5005	MLflow tracking + model registry
prometheus	9090	Metrics scraping every 15s
grafana	3001	12-panel NRT dashboard
alertmanager	9093	Gmail SMTP alert routing
airflow	8080	Pipeline orchestration DAG UI
node_exporter	9100	System CPU/memory/disk metrics

FastAPI Swagger UI

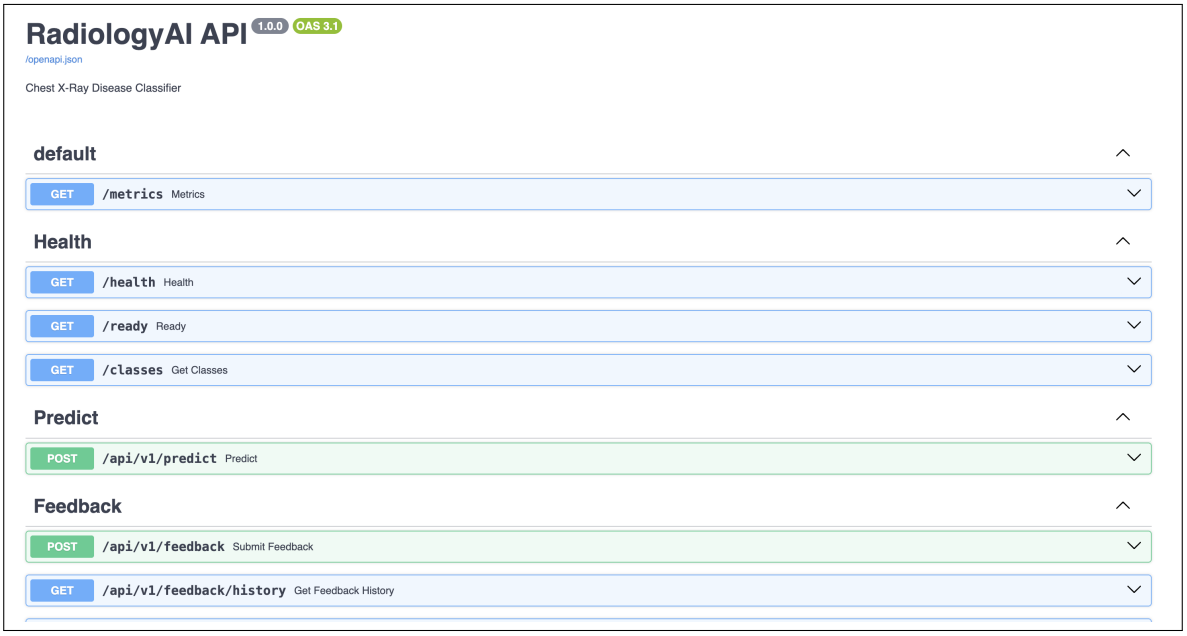


Figure 15: FastAPI auto-generated Swagger UI at /docs showing all 12 REST endpoints organised by tag: Health, Predict, Feedback, Reports, Monitoring. Each endpoint shows full request/response schema.

Software Engineering

Design Principles

- **Loose coupling** — React and FastAPI are fully independent. API URL via `REACT_APP_API_URL` env variable.
- **Config-driven** — All secrets, paths, thresholds in `.env` via Pydantic Settings. Zero hardcoded values.
- **OOP paradigm** — Pydantic model classes, service classes, router classes.
- **PEP8 compliance** — enforced by flake8 in GitHub Actions CI.
- **Logging** — Python logging on all endpoints. Every prediction, feedback, drift check logged to `logs/`.
- **Exception handling** — HTTPException on all routes. try/catch in all services. React error boundaries.
- **Inline documentation** — Google-style docstrings on all Python functions.

Testing

Category	Total	Passed	Failed	Pass Rate	
Health & System	4	4	0	100%	
Prediction Endpoint	5	5	0	100%	
Feedback System	4	4	0	100%	
Drift & Reports	4	4	0	100%	
Monitoring	3	3	0	100%	
TOTAL	**20**	**20**	**0**	**100%**	

Challenges & Solutions

Challenge	Solution
Class imbalance (73% Pneumonia)	Weighted cross-entropy loss + Albumentations augmentation
MLflow DB version mismatch	Upgraded local MLflow; used named Docker volume for DB
DVC accidentally deleted datasets	Re-downloaded from Kaggle; fixed .dvcignore
Airflow container path errors	Mounted project as volume; set RADIOLOGYAI_BASE env var
Feedback gauge reset on restart	load_feedback_from_file() on startup restores gauge from JSON
Grafana disk panel empty on macOS	Switched to fstype filter — node_exporter paths differ on Mac
Docker build including venv (1.2GB)	Added .dockerignore — build time reduced 10x
EfficientNetB0 weights download failure	Copied pretrained weights from GPU server via scp

Ethical Considerations

- **Clinical disclaimer** — every prediction: *"AI-assisted preliminary assessment only. Not a substitute for professional diagnosis."*
- **Data privacy** — X-ray images not stored permanently. Processed for inference only.
- **No cloud** — fully on-premise deployment. No medical data transmitted externally.

- **Bias awareness** — class imbalance addressed. Per-class sensitivity ¿ 97%.
- **Transparency** — Grad-CAM shows exactly which lung regions the AI focused on.

Conclusion

RadiologyAI demonstrates the practical application of AI and MLOps principles to healthcare's most pressing challenges. The system achieves:

- **99.09% validation accuracy** and **val F1 = 0.9871** on EfficientNetB0
- **142ms inference latency** — within the 200ms clinical SLA
- Complete MLOps lifecycle: DVC (9-stage DAG) → MLflow (3 experiments + registry) → Airflow (feedback-triggered retraining) → Prometheus/Grafana (12-panel dashboard) → Alertmanager (Gmail notifications)
- Clinician-facing React UI with 6 pages including drift detection, model comparison, patient history, and downloadable PDF clinical reports
- A closed feedback loop where radiologist confirmations drive automated model retraining via Airflow

"Technology should serve the clinician who serves the patient — RadiologyAI is a step in that direction."